

BD-theory2-resumen.pdf



biingus



Bases de Datos



2º Grado en Ingeniería Informática del Software



**Escuela de Ingeniería Informática
Universidad de Oviedo**

FUNCTIONAL DEPENDENCIES

CUSTOMER (DNI, name, city, PC, province)

PC → province (postal code implies a specific province)
 DNI → name
 DNI → city
 DNI → name, city, PC, province

PC → province
 DNI → name, city, PC, province
 PC → province
 DNI → name, city, PC
 F₁ = {PC → province, DNI → name, city, PC, province}
 F₂ = {PC → province, DNI → name, city, PC}
 F₃ = {DNI → name, city, PC, province}

We can delete province because DNI → PC and PC → province, so DNI → province is unnecessary. TRANSITIVITY

MINIMAL COVER (F₂)

MINIMAL COVER (F₂)

- Set that is equivalent to other, without redundant dependencies.

EXTRAPOUS ATTRIBUTE: Attributes that can be deleted and the set is still equivalent.

ARMSTRONG RULES

1. REFLEXIVITY: $A \rightarrow B \implies B \rightarrow A$
2. AUGMENTATION: $A \rightarrow B \implies AB \rightarrow AB$
3. TRANSITIVITY: $A \rightarrow B, B \rightarrow C \implies A \rightarrow C$
4. DECOMPOSITION: $A \rightarrow BC, A \rightarrow B \implies A \rightarrow C$
5. UNION: $A \rightarrow B, A \rightarrow C \implies A \rightarrow BC$
6. PSEUDOTRANSITIVITY: $A \rightarrow B, BC \rightarrow C \implies AB \rightarrow C$

REDUNDANCIES

CUSTOMER (DNI, name, city, PC, province)

1. Paco OVD 33009 AST
2. Carmen OVD 33009 LEON

DNI=1 is associated with Paco
 DNI=1 is associated with AST
 33009 is associated with LEON
 33009 is associated with AST
 Forced to always repeat the same values
 REDUNDANCY

* If left side can appear multiple times → ONLY KEYS IN LEFT SIDE

DNI → name, city, PC
 PC → province
 PC is not part of a key: NEW TABLE where PC is key:
 Customer (DNI, name, city, PC)
 Postal_Codes (PC, province)
 DECOMPOSITION

NORMAL FORMS

- The higher the NF, less redundancy

BCNF (Boyce-Codd NF)

1. $X \rightarrow Y$ is trivial
2. X is a SUPERKEY

* STEPS

R = {A, B, C, D, E, F, G, H}

F = {AB → C, C → DE, E → FH}

KEY: ABCG

C, E NOT KEYS

E → FH

NO DEPENDENCIES → PK

R₁ = {E, F, G}

R₂ = {C, D, E}

R₃ = {A, B, C}

R₄ = {A, B, G}

DECOMPOSITION

- Divide a table into smaller ones to avoid REDUNDANCIES

- $r(R_1) \times r(R_2) \times \dots \times r(R_n) \rightarrow r = r_1 \times r_2 \times \dots \times r_n$

LOSSLESS JOIN: if you join the derived tables you get the original one.

R = {DNI, name, city, PC}

R₁ = {PC, province}

F₁ = {DNI → name, city, PC, province}

F₂ = {PC → province}

* JOINING ATTRIBUTES SHOULD BE KEYS TO AVOID LOSSY JOINS

* Needs to be stated because it not longer appears in the table

R = {customer, branch, advisor}

F = {customer branch → advisor, advisor → branch}



C IS ISOLATED: WILL BE PART OF ALL KEYS

KEYS: CA, CB

F = {C → A}

XA → B

NOT A SUPERKEY! (only part of one)

R₁ = {A, B}

R₂ = {C, A}

F₁ = {A → B}

F₂ = {C → A}

NO DEPENDENCIES → PK

DEPENDENCY PRESERVING

R = {A, B, C}

F = {A → B, B → C, C → A}

R₁ = {A, B}

R₂ = {B, C}

F₁ = {A → B, B → A}

F₂ = {B → C, C → B}

Not originally in F

By union (F₁ ∪ F₂) you get C → A

C → B ∨ B → A = C → A

3NF

* Decompositions are not DEPENDENCY PRESERVED

PROPERTIES

1. $X \rightarrow Y$ is trivial
2. X is SUPERKEY
3. All Y are PRIME → PART OF KEY



R = {customer, branch, advisor, office}

F = {CB → A, A → BO}



R₁ = {C, B, A}

R₂ = {A, B, O}

F₁ = {CB → A, A → B}

F₂ = {A → BO}

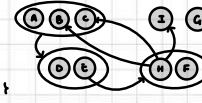
ALGORITHM

0. F should be MINIMAL COVER
1. Each dependency gets a TABLE
2. Delete (if possible) repeated dependencies.
3. If no candidate keys of original decomposition
 CREATE TABLE with those attributes

EXERCISE

R = {A, B, ..., I}

F = {ABC → DE, E → HF, H → IBC}



* NO INCOMING ARROWS → PART OF KEY: A, G

* KEYS: BCAG, HAG, EAG

1. TABLE PER RELATION

3NF R₁ = {A, B, C, D, E}

BCNF R₂ = {E, F, H}

BCNF R₃ = {H, I, B, C}

BCNF (S) R₄ = {A, B, G}

F₁ = {ABC → DE, E → BC}

F₂ = {E → HF}

F₃ = {H → IBC}

F₄ = {I} CANDIDATE KEY: NO DEPENDENCIES

2. REPEATED DEPENDENCIES

3. CANDIDATE KEY in F? G is MISSING